

Submitted to *Econometrica*

DOCUMENTACIÓN TÉCNICA DETALLADA DEL MOTOR DE SIMULACIÓN APP_DL.PY: REGRETNET, LAGRANGIANO AUMENTADO Y GENERACIÓN DE LA TABLA 5.2

HUMBERTO BERNAL

Universidad Colegio Mayor de Cundinamarca

Este documento describe, a nivel de código y de fórmula, cómo la aplicación Streamlit `app_DL.py` transforma los parámetros elegidos por el usuario en la trayectoria simulada que compone la Tabla 5.2 del artículo principal. La versión actual del motor reemplazó por completo el solver analítico cerrado de versiones anteriores por una red neuronal de política (`StateNetwork`) genuinamente entrenada en PyTorch mediante descenso de gradiente (Adam) y un esquema de Lagrangiano Aumentado que penaliza el arrepentimiento (*regret*) de incentivos, al estilo de la literatura de *Differentiable Economics* (RegretNet). Se detalla (i) la arquitectura de archivos, (ii) el cálculo de priors dinámicos y el mapeo de la barra lateral al estado inicial, (iii) la fase de entrenamiento del planificador social, (iv) los pasos del bucle de simulación período a período que usa la red ya entrenada para inferir la política óptima, (v) el ensamblaje de la Tabla 5.2 (ahora transpuesta) y (vi) las visualizaciones de diagnóstico, incluyendo la curva de convergencia del entrenamiento. Se señalan además dos particularidades de implementación no evidentes a primera vista: el entrenamiento ocurre sobre un único punto de estado fijo repetido en cada época, y la verificación de IC^K dentro del bucle de simulación es un marcador de posición que siempre se satisface trivialmente.

KEYWORDS: deep learning, RegretNet, mechanism design, Lagrangiano aumentado, simulación.

Nota de vigencia (añadida posteriormente). Las Secciones 3–4 documentan el motor `StateNetwork/RegretNet` de `dl_mechanism.py`. Se deja constancia de que, a la fecha, **`dl_mechanism.py` no está importado en ningún punto de `app_DL.py`** — ese código no se ejecuta en la aplicación actual. La Sección 6 documenta, de forma independiente y autocontenida, el mecanismo que sí está activo hoy en la pestaña 3 (*Results*): la optimización dinámica del Estado en $\tau = 1$ mediante `train_captor_value_net.py`.

1. ARQUITECTURA DE ARCHIVOS

El motor de simulación se compone de los siguientes archivos bajo `Streamlit/Deep_Learning/`:

Archivo	Rol	¿Se ejecuta?
app_DL.py	Interfaz Streamlit; calcula priors, entrena la red de política y corre el bucle de simulación período a período	Es el punto de entrada
dl_mechanism.py	PARAMS_ORGANIZACION; clases KidnappingEnvironment (riesgos de Cox + madurez), StateNetwork (red de política, torch.nn.Module), MDGLayer (temperatura y sorteo MDG), RegretNetTrainer (entrenador con Lagrangiano Aumentado)	Las cuatro clases se instancian y se usan activamente
dl_engine.py	BackwardInductionMLP, run_backward_induction_dl (emulador NumPy de una versión previa)	No se importa desde app_DL.py — módulo huérfano heredado de una iteración anterior del proyecto

Nota metodológica. A diferencia de una versión previa de este mismo archivo — donde la rama “Deep Learning” era en realidad un solver analítico cerrado sin ninguna red neuronal entrenada de por medio —, la implementación actual sí ejecuta un entrenamiento genuino: `RegretNetTrainer` hace *backpropagation* y pasos de `optimizer.step()` sobre los parámetros de `StateNetwork` en cada corrida de la app (§3). Con todo, subsiste una particularidad importante: como se explica en §3.5, el entrenamiento se realiza sobre un *único* punto del espacio de estados (creencia inicial μ_0 y $t = T_{\text{horizon}}/2$ fijos), repetido en cada época, y no sobre trayectorias variadas del juego. La red generaliza después a estados nunca vistos durante el entrenamiento cuando se usa para inferencia en el bucle de simulación (§4).

2. ENTRADA DE DATOS: DE LA BARRA LATERAL AL ESTADO INICIAL

Todo el “dato” que alimenta la simulación es sintético: no se carga ningún archivo externo en este módulo. Los controles del usuario parametrizan (i) la creencia previa μ_0 , ya sea de forma fija, manual, o *model-driven* vía un logit multinomial de covariables, y (ii) los hiperparámetros del entorno, de la capa MDG y del entrenador RegretNet.

2.1. Priors dinámicos model-driven

Cuando el escenario es *Custom* y el modo de prior es *Model-Driven*, la función `calculate_dynamic_priors(region, victim_profile)` construye la creencia inicial como un logit multinomial sobre covariables observables de tipo, región y perfil de víctima:

$$s(\theta) = \delta(\theta) + \eta_z(\theta) + \xi_v(\theta), \quad \mu_0(\theta) = \frac{e^{s(\theta)}}{\sum_{\theta'} e^{s(\theta')}} \quad (1)$$

donde $\delta(\theta)$ es un efecto fijo por tipo (COEF_DELTA: FARC= 0, ELN= −0,63, PAR= −0,03, DC= −1,25), $\eta_z(\theta)$ es un efecto de región geográfica z (COEF_ETA, igual para los 4 tipos dentro de cada región: Metropolitan= 0, Andean= −0,45, Caribbean= −0,70,

Pacific/Zone Red= $-0,20$, East/Jungle= $-0,32$), y $\xi_v(\theta)$ es un efecto del perfil de la víctima (COEF_XI: Public= $1,36$, Private= 0 , igual para los 4 tipos). La ecuación (1) es un softmax estándar sobre estos tres efectos aditivos — nótese que η_z y ξ_v no varían por tipo dentro de cada nivel de la covariable, por lo que en la práctica solo $\delta(\theta)$ diferencia a los cuatro tipos; región y perfil de víctima desplazan la escala pero no la forma relativa entre tipos.

2.2. Controles de la barra lateral

Control	Variable	Efecto
escenario (select-box)	tipo_real, priors_init,...	Escenario I (ELN) y II (PAR) fijan tipo real, priors, perfil y todos los hiperparámetros de calibración; <i>Custom</i> habilita todos los controles manuales
prior_mode_val (solo en Custom)	<i>Model-Driven</i> vs. <i>Manual (Sliders)</i>	<i>Model-Driven</i> usa la ecuación (1); <i>Manual</i> usa 4 sliders normalizados a 100%
T_horizon, ψ_H , $\underline{c}$ (sliders)	T_horizon, ψ_H , $\underline{c_bar}$	Horizonte temporal, peso de exploración activa, piso de temperatura MDG
Expander <i>Model Calibration</i>	$\alpha_0, \gamma_0, T_{\text{mad}}, \lambda_4, T_0, \eta_{\text{cal}}$	Alimentan KidnappingEnvironment (madurez, tasa exógena de liberación) y MDGLayer (temperatura inicial y decaimiento)
Expander <i>RegretNet Hyperparameters</i>	learning_rate, rho_lagrangian, epochs_training	Tasa de aprendizaje de Adam, coeficiente de penalización cuadrática ρ , número de épocas de entrenamiento
botón <i>Start Simulation</i>	st.session_state. simulation_run	Ver nota de comportamiento abajo

Comportamiento del botón *Start Simulation*. A diferencia de una versión previa, la variable de control ahora vive en `st.session_state.simulation_run`, inicializada a `True` la primera vez que se carga la página (`app_DL.py`, líneas 250–251), antes de que el usuario presione ningún botón. En consecuencia, la simulación completa (entrenamiento + bucle) se ejecuta automáticamente en cada carga o *rerun* de la app; el botón simplemente reafirma el mismo valor `True`. El efecto práctico es que basta con cambiar cualquier slider para disparar un nuevo entrenamiento y una nueva simulación, sin necesidad de pulsar el botón.

2.3. Inicialización del estado

$$p_{\text{init}} = \frac{\text{priors_init}}{\sum \text{priors_init}}, \quad \mu_0(\theta) = p_{\text{init}}[\theta], \quad \theta \in \{DC, PAR, ELN, FARC\} \quad (2)$$

con parámetros de escala fijos $R_{\text{base}} = 100$, $\omega_p = 0,5$ (ponderador del costo de transferencia), $\omega_k = 2,0$ (ponderador del costo de pérdida de vida), y generador `rng = np.random.default_rng(1007)` para todos los sorteos estocásticos del bucle de simulación (determinístico dado el mismo conjunto de inputs).

3. FASE DE ENTRENAMIENTO: REGRETNET CON LAGRANGIANO AUMENTADO

Antes de correr la simulación período a período, la app entrena una red neuronal de política que representa al planificador social (el Estado). Esta es la novedad central respecto a versiones anteriores del motor.

3.1. Arquitectura de StateNetwork

$$\text{entrada (7)} \xrightarrow{\text{Linear+ReLU}} 32 \xrightarrow{\text{Linear+ReLU}} 32 \xrightarrow{\text{Linear+Sigmoid}} (\alpha_t, \gamma_t) \in [0, 1]^2 \quad (3)$$

con vector de entrada

$$\text{state} = (\mu_t(DC), \mu_t(PAR), \mu_t(ELN), \mu_t(FARC), \theta_F, \theta_V, t/150) \quad (4)$$

donde $\theta_F \in \{0, 1\}$ codifica *Family Payment Capacity* (*High*= 1) y $\theta_V \in \{0, 1\}$ codifica *Victim Profile* (*Public*= 1). La activación sigmoide en la capa de salida garantiza $(\alpha, \gamma) \in [0, 1]^2$ sin necesidad de proyección posterior (a diferencia del solver analítico previo, que sí requería recorte explícito de bordes).

3.2. Pérdida social esperada

Para un estado de entrada dado, la red produce una política honesta $(\alpha_h, \gamma_h) = \text{net}(\text{state_in})$. La pérdida social ponderada por la creencia actual es

$$\mathcal{L}_{\text{social}} = \sum_{\theta} \mu_t(\theta) \left[\underbrace{\omega_p R_{\text{base}}(1 - \alpha_h) + \omega_k \lambda_{\text{kill}}(\theta)(1 + \gamma_h)}_{\text{costo de transferencia + riesgo de muerte esperado}} + \underbrace{c_{\alpha}(\theta) \alpha_h^2 + c_{\gamma}(\theta) \gamma_h^2 + 0,2(c_{\alpha}(\theta) + c_{\gamma}(\theta)) \alpha_h \gamma_h}_{C_{\text{maint}}(\theta): \text{costo de mantenimiento cuadrático}} \right] \quad (5)$$

donde el último término $\psi_H H(\mu_t)$ es el subsidio de exploración: penaliza la pérdida en proporción a la entropía actual de la creencia, incentivando indirectamente que el entrenamiento favorezca políticas que — a través del término de arrepentimiento descrito abajo — ayuden a reducir la incertidumbre.

3.3. Arrepentimiento empírico (regret) y restricciones IC

Para cada tipo verdadero candidato θ , se calcula la utilidad del secuestrador si reporta honestamente frente a la mejor utilidad posible si mintiera reportando otro tipo $\theta' \neq \theta$:

$$u_{\text{honesto}}(\theta) = \lambda_{\text{pay}}(\theta) R_{\text{base}}(1 - \alpha_h) - c_{\gamma}(\theta) \gamma_h \quad (6)$$

$$u_{\text{mentira}}(\theta) = \max_{\theta' \neq \theta} \left\{ \lambda_{\text{pay}}(\theta) R_{\text{base}}(1 - \alpha_h(\theta')) - c_{\gamma}(\theta) \gamma_h(\theta') \right\} \quad (7)$$

donde $(\alpha_h(\theta'), \gamma_h(\theta'))$ es la política que la red produciría si el Estado observara una creencia degenerada $\mu = \delta_{\theta'}$ (reporte de tipo θ' con certeza total). El arrepentimiento por tipo es

$$\text{Regret}(\theta) = \max\{0, u_{\text{mentira}}(\theta) - u_{\text{honesto}}(\theta)\} \quad (8)$$

Esta construcción — evaluar la red en reportes contrafactuales y penalizar cuando mentir es más rentable que ser honesto — es exactamente el mecanismo de *RegretNet (Differentiable Economics)* adaptado a este entorno de 4 tipos.

3.4. Lagrangiano Aumentado y ascenso dual

La pérdida total que se retropropaga combina la pérdida social con una penalización sobre el arrepentimiento de cada tipo, usando multiplicadores de Lagrange $\lambda_\theta \geq 0$ actualizados por ascenso dual:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{social}} + \sum_{\theta} \lambda_{\theta} \text{Regret}(\theta) + \frac{\rho}{2} \sum_{\theta} \text{Regret}(\theta)^2 \quad (9)$$

$$\text{(primal)} \quad \text{Adam step sobre los pesos de StateNetwork minimizando (9)} \quad (10)$$

$$\text{(dual)} \quad \lambda_{\theta} \leftarrow \max\{0, \lambda_{\theta} + \rho \text{Regret}(\theta)\} \quad (11)$$

El paso dual ocurre *después* de cada `optimizer.step()`, incrementando el multiplicador de un tipo cada vez que su restricción IC se viola, lo cual — en la siguiente época — encarece más ese arrepentimiento en la pérdida total. Este es el patrón estándar del método de Lagrangiano Aumentado para imponer restricciones de desigualdad en optimización diferenciable.

3.5. Bucle de entrenamiento y su particularidad

Particularidad importante. Nótese que `mu_t` y `t` permanecen **constantes** durante todo el bucle de entrenamiento: `mu_t` es la creencia inicial μ_0 (aún no se ha actualizado, pues la actualización bayesiana ocurre más adelante, dentro del bucle de simulación) y `t` está fijo en $T_{\text{horizon}}/2$. Es decir, el entrenamiento consiste en `epochs_training` pasos de descenso de gradiente sobre un único punto del espacio de entrada, no sobre un conjunto de trayectorias o estados muestreados. La red converge a una buena política *para ese estado particular* (y, por construcción del término de *regret*, también aprende algo sobre los estados degenerados $\delta_{\theta'}$ usados como reportes contrafactuales). Cuando posteriormente se usa la red para inferencia en el bucle de simulación (§4) con creencias μ_t y períodos t que sí varían y se alejan del punto de entrenamiento, la política inferida es una **extrapolación** de la red fuera de su régimen de entrenamiento explícito, no una política verificada en esos estados. Esto no invalida el ejercicio — ReLU + Sigmoid con solo 2 capas ocultas tiende a generalizar suavemente cerca del punto de entrenamiento — pero es una limitación metodológica a tener en cuenta al interpretar la Tabla 5.2 como una política óptima verificada en todo el horizonte, y no solo en $t = T_{\text{horizon}}/2$.

La interfaz muestra en vivo, época a época, la pérdida social y el arrepentimiento total agregado (`progress_bar`, `status_text`), y al finalizar reporta el tiempo total de entrenamiento en milisegundos.

4. EL BUCLE DE SIMULACIÓN: `FOR T IN RANGE (T_HORIZON)`

Una vez entrenada la red, se instancian `env = KidnappingEnvironment (T_mad, lambda_4)` y `mdg = MDGLayer (T0, c_bar, eta_cal)`, y arranca el bucle que sí varía μ_t y t período a período, produciendo una fila del DataFrame final por iteración.

4.1. Paso 1: Filtro de madurez

$$M(t) = \min \left\{ 1, \left(\frac{t}{T_{\text{mad}}} \right)^2 \right\} \quad (12)$$

Multiplicador que escala las intensidades de riesgo (paso 8) desde 0 hasta 1 a medida que el caso “madura” operativamente; alcanza su tope de 1 cuando $t \geq T_{\text{mad}}$.

4.2. Paso 2: Entropía y precisión de la creencia

$$H(\mu_t) = - \sum_{\theta} \mu_t(\theta) \ln \mu_t(\theta), \quad H(\mu_0) = - \sum_{\theta} \mu_0(\theta) \ln \mu_0(\theta) \quad (13)$$

$$\iota_t = \max_{\theta} \mu_t(\theta), \quad \hat{\theta}_t = \arg \max_{\theta} \mu_t(\theta) \quad (14)$$

4.3. Paso 3: Temperatura MDG (con escala T_0)

$$T_t = T_0 \cdot \max \left\{ \frac{H(\mu_t)}{H(\mu_0)} e^{-\eta_{\text{cal}} t}, \underline{c} \right\} \quad (15)$$

A diferencia de una versión previa (donde el piso $\underline{c}$ actuaba directamente sobre la temperatura sin escala), aquí la temperatura inicial T_0 multiplica todo el término, permitiendo calibrar tanto el nivel base de ruido MDG como su piso asintótico.

4.4. Paso 4: Inferencia de política vía la red entrenada

$$(\alpha_t^*, \gamma_t^*) = \text{net}(\mu_t(DC), \mu_t(PAR), \mu_t(ELN), \mu_t(FARC), \theta_F, \theta_V, t/150) \quad (16)$$

evaluado con `torch.no_grad()` (inferencia pura, sin retropropagación). Este paso reemplaza por completo al solver cuadrático analítico de versiones anteriores: la política óptima del Estado en cada período es ahora una *forward pass* de la red ya entrenada en §3.

4.5. Paso 5: Rescate vs. Negociación

$$p_{\text{surv}} = \frac{1}{1 + \exp(-(\text{alpha_leth}(\theta_K^*) + 1,2 \iota_t \mathbf{1}\{\hat{\theta}_t = \theta_K^*\}))} \quad (17)$$

$$V_{\text{rescate}} = \omega_k(1 - p_{\text{surv}}) + 1,5 (\gamma_t^*)^2 \quad (18)$$

$$V_{\text{negociar}} = \omega_p R_{\text{base}}(1 - \alpha_t^*) + \omega_k \lambda_{\text{kill}}(\theta_K^*)(1 + \gamma_t^*) \quad (19)$$

Si $V_{\text{rescate}} < V_{\text{negociar}}$: $a_S^* = \text{“Rescue”}$, forzando $\alpha_t^* \leftarrow 0$, $\gamma_t^* \leftarrow 0,9$; en otro caso, $a_S^* = \text{“Negotiate”}$. La probabilidad de supervivencia p_{surv} ahora se modela con una función logística explícita (antes era una expresión condicional *ad hoc*).

4.6. Paso 6: Mejor respuesta del secuestrador real y de la familia

Idéntica en forma a versiones anteriores, evaluada con los parámetros del tipo verdadero θ_K^* y las políticas (α_t^*, γ_t^*) recién inferidas:

$$u_{\text{cont}} = \lambda_{\text{pay}}(\theta_K^*) R_{\text{base}}(1 - \alpha_t^*) - c_\gamma(\theta_K^*) \gamma_t^*, \quad u_{\text{kill}} = -\text{alpha_leth}(\theta_K^*), \quad (20)$$

$$u_{\text{rel}} = -k_{\text{rel}}(\theta_K^*), \quad a_K^* = \arg \max\{u_{\text{cont}}, u_{\text{kill}}, u_{\text{rel}}\} \quad (21)$$

$$u_{\text{coop}} = 0,8 R_{\text{base}} - 0,5 \gamma_t^*, \quad u_{\text{col}} = 0,5 R_{\text{base}} - 2,0 \alpha_t^*, \quad a_F^* = \begin{cases} \text{Cooperate} & u_{\text{coop}} > u_{\text{col}} \\ \text{Collude} & \text{en otro caso} \end{cases} \quad (22)$$

4.7. Paso 7: Sorteo MDG

`mdg.sample_action(intent, alternatives, temp_t, rng)` implementa la misma ley logit de implementación MDG que en versiones anteriores, ahora encapsulada como método de `MDGLayer`:

$$\mathbb{P}(\tilde{a} = a \mid a^*, T_t) = \frac{\exp(\mathbf{1}\{a = a^*\}/T_t)}{\sum_{a'} \exp(\mathbf{1}\{a' = a^*\}/T_t)} \quad (23)$$

aplicada independientemente a las tres decisiones del Estado, el secuestrador y la familia, produciendo $\tilde{a}_S, \tilde{a}_K, \tilde{a}_F$ (acciones *realizadas*) frente a a_S^*, a_K^*, a_F^* (acciones *intencionadas*).

4.8. Paso 8: Riesgos en competencia con filtro de madurez

`env.compute_cox_hazards(alpha, gamma, theta, t)` calcula, para el tipo dado:

$$\lambda_{\text{pago}}(\theta) = \lambda_{\text{pay}}(\theta) (1 - \alpha) M(t), \quad \lambda_{\text{muerte}}(\theta) = \lambda_{\text{kill}}(\theta) (1 + \gamma) M(t), \quad (24)$$

$$\lambda_{\text{rescate}}(\theta) = \lambda_{\text{res}}(\theta) (1 + \gamma) M(t), \quad \lambda_{\text{liberación}} = \lambda_4 \quad (\text{constante, sin madurez}) \quad (25)$$

A diferencia de versiones previas, las tres primeras tasas ahora se escalan por el filtro de madurez $M(t)$ (§4, Paso 1): al inicio del caso ($t \ll T_{\text{mad}}$) todas las intensidades de resolución se atenúan, reflejando que la organización aún no ha “madurado” operativamente; la tasa de liberación exógena λ_4 no se ve afectada. Con $p_{\text{Cont}} = \exp(-\sum_j \lambda_j)$, se sortea si el caso absorbe (termina) este período, igual que en versiones anteriores.

4.9. Paso 9: Actualización bayesiana de creencias

Idéntica en estructura a versiones previas, mas ahora usando `env.compute_cox_hazards(...)` evaluado en cada tipo candidato θ para calcular la verosimilitud:

$$\mu_{t+1}(\theta) = \frac{\mu_t(\theta) \text{lik}_t(\theta)}{\sum_{\theta'} \mu_t(\theta') \text{lik}_t(\theta')}, \quad \text{lik}_t(\theta) = \begin{cases} p_{\text{Cont}}(\theta) & \text{outcome} = \text{Continue} \\ \lambda_{\text{outcome}}(\theta) & \text{en otro caso} \end{cases} \quad (26)$$

con reinicio a la uniforme 0,25 si el denominador colapsa numéricamente ($< 10^{-12}$).

4.10. Paso 10: Restricciones ex-post — y una particularidad en IC^K

Se registran tres indicadores de cumplimiento:

- IR^K : $u_{\text{cont}} \geq -1,5$ (el umbral cambió de -1 a $-1,5$ respecto a versiones anteriores).
- IR^F : $u_{\text{coop}} \geq u_{\text{col}}$.
- IC^K : calculado mediante

Particularidad en la verificación de IC^K . El comentario en el código señala expresamente que esta expresión está “simplificada para evaluación”. En efecto, `p_opt` es siempre `PARAMS_ORGANIZACION[tipo_real]` — los parámetros del tipo *verdadero* — independientemente de la variable de bucle `other`; la expresión dentro del `max` no depende de `other` en absoluto y es idéntica, término a término, a la definición de `u_cont` ya calculada arriba. En consecuencia, `u_lying_max` termina siendo exactamente igual a `u_cont`, y `ic_k_ok = (u_cont >= u_cont)` es **siempre verdadero** por construcción, sin importar la política inferida. A diferencia del *regret* genuino calculado durante el entrenamiento (§3, que sí evalúa la red en reportes contrafactuales), esta verificación ex-post dentro del bucle de simulación es un marcador de posición (*placeholder*) que no prueba nada sobre incentivos — la columna `IC_K` en la Tabla 5.2 mostrará siempre “Complies”.

Finalmente se agrega la fila al registro, con `Calculo_ms` midiendo únicamente el tiempo de la inferencia `net(state_in)` del Paso 4 (no el resto de los cálculos del período).

5. GENERACIÓN DE LA TABLA 5.2

1. `df_sim = pd.DataFrame(t_records)` — una fila por período simulado (T_{horizon} filas en total).
2. **Métricas de desempeño** (tarjetas superiores de la pestaña 2): tiempo total de simulación, tiempo promedio de inferencia por paso, y el factor de aceleración (*speedup*) respecto a un límite de referencia de 2 minutos:

$$\text{Speedup} = \frac{120\,000 \text{ ms}}{\text{tiempo total (ms)}} \quad (27)$$

3. **Formato de 3 decimales:** las columnas de creencias e instrumentos ($\mu_{DC}, \mu_{PAR}, \mu_{ELN}, \mu_{FARC}, \alpha^*, \gamma^*$) se formatean como cadenas de texto con `f"{x: , .3f}"`.
4. **Renombrado descriptivo** de columnas (p.ej. `alpha*` $\rightarrow$ “Interdiction (α^*)”, `mu_DC` $\rightarrow$ “Belief DC (μ_{DC})”).
5. **Transposición:** `df_t52_display.set_index("Period (t)").T` — a diferencia de versiones previas donde los períodos eran *filas*, ahora cada período es una **columna** y cada variable (creencias, instrumentos, acciones) es una **fila**. Se muestran las primeras 15 columnas (`.iloc[:, :15]`), es decir, los primeros 15 períodos, igual que antes, pero en formato transpuesto.

Determinismo. La tabla se reconstruye por completo en cada *rerun* de la app (incluyendo un nuevo entrenamiento de la red, §3). Con `rng = np.random.default_rng(1007)` fijo para el bucle de simulación, la parte estocástica es reproducible; sin embargo, dado que `StateNetwork` se reinicializa con pesos aleatorios de PyTorch (sin semilla fija establecida explícitamente para `torch`) y se re-entrena desde cero en cada *rerun*, la política (α_t^*, γ_t^*) — y por tanto la tabla completa — **puede variar ligeramente entre ejecuciones** con los mismos inputs, a diferencia del comportamiento enteramente determinístico del solver analítico de versiones anteriores.

6. OPTIMIZACIÓN DINÁMICA DEL ESTADO EN $\tau = 1$ (PESTAÑA 3, BOTÓN *RUN STATE OPTIMIZATION*)

Esta sección documenta el mecanismo realmente activo hoy en `app_DL.py`, disparado por el botón “*Run State Optimization ($\tau = 1$, dynamic)*” en la parte superior de la pestaña 3. Toda la lógica de optimización vive en el script independiente `train_captor_value_net.py` (función `solve_state_problem`), que `app_DL.py` importa directamente; la red de valor se entrena *offline* (no dentro de la sesión de Streamlit) y se carga ya entrenada desde `captor_value_net_T10.pt`.

6.1. El problema formal y la penalización de anclaje

Para $\tau = 1$ (y de manera general para cualquier período τ dentro del bucle de simulación de la pestaña 3), dado el posterior μ_τ (calculado secuencialmente a través de la regla de Bayes), el Estado resuelve la política óptima para cada rama de negociación o rescate $b \in \{\text{Rescate}, \text{Negociar}\}$ resolviendo:

$$V_\tau^b(\mu_\tau) = \min_{(\alpha, \gamma) \in [0,1]^2 : \Gamma_\tau(\mu_\tau)} \left\{ \sum_{\theta} \mu_\tau(\theta) L_\tau^b(\theta, \alpha, \gamma) + \Pi_b^S(\alpha, \gamma; \mu_\tau) - \psi_H \Delta H_\tau(\alpha, \gamma) \right\}, \quad (28)$$

donde el problema se resuelve por **búsqueda exhaustiva en malla** 101×101 sobre la grilla $(\alpha, \gamma) \in [0,1]^2$ (GA, `GG = np.meshgrid(np.linspace(0,1,101), np.linspace(0,1,101))`). Nótese que la política no se obtiene por descenso de gradiente directo ni por evaluación forward de una red de política; la red entrenada únicamente aproxima el valor de continuación futuro $V_{\text{cont}}(\theta, \alpha, \gamma)$ (ec. 46), el cual alimenta el término de costo de mantenimiento en L_τ^N . Tras resolver ambas ramas, el Estado elige la acción que minimice el valor esperado global: $a_\tau^{S*} = \arg \min_b \{V_\tau^R, V_\tau^N\}$.

Penalización de anclaje (Prima de desviación) $\Pi_b^S(\alpha, \gamma; \mu_\tau)$

Para asegurar la convergencia y regularidad de los controles óptimos (α_t^*, γ_t^*) , el término $\Pi_b^S(\alpha, \gamma; \mu_\tau)$ impone una penalización cuadrática por desviaciones respecto a los benchmarks de información perfecta ponderados por creencia:

$$\Pi_b^S(\alpha, \gamma; \mu_\tau) = \chi_\alpha (\alpha - \alpha_b^\mu(\mu_\tau))^2 + \chi_\gamma (\gamma - \gamma_b^\mu(\mu_\tau))^2, \quad (29)$$

donde los coeficientes de penalización están calibrados en $\chi_\alpha = 0.05$ y $\chi_\gamma = 0.05$. Los centros de anclaje $\alpha_b^\mu(\mu_\tau)$ y $\gamma_b^\mu(\mu_\tau)$ se determinan dinámicamente en cada período a partir de los benchmarks individuales de tipo puro $\alpha_b^{\theta,*}$ y $\gamma_b^{\theta,*}$ ponderados por la creencia actual del Estado:

$$\alpha_b^\mu(\mu_\tau) = \sum_{\theta \in \Theta} \mu_\tau(\theta) \alpha_b^{\theta,*}, \quad (30)$$

$$\gamma_b^\mu(\mu_\tau) = \sum_{\theta \in \Theta} \mu_\tau(\theta) \gamma_b^{\theta,*}, \quad (31)$$

para cada rama $b \in \{\text{Rescate}, \text{Negociar}\}$.

Benchmarks individuales bajo información perfecta

Para cada tipo puro de captor $\theta \in \Theta$, el Estado precalcula los benchmarks óptimos bajo certeza informacional. Estos benchmarks corresponden a los minimizadores de la función de pérdida sin subsidio entrópico ($\psi_H = 0$) y bajo la creencia degenerada $\mu_\theta = 1$:

- **Rama Rescate ($b = \text{Rescate}$):**

$$(\alpha_R^{\theta,*}, \gamma_R^{\theta,*}) = \arg \min_{(\alpha, \gamma) : \Gamma_\tau(\mathbf{1}_\theta)} \{ \omega_k (1 - p_{\text{surv}}(\theta, \theta)) + C_{\text{ops}}(\gamma, \alpha; \theta) \} \quad (32)$$

- **Rama Negociación ($b = \text{Negociar}$):**

$$(\alpha_N^{\theta,*}, \gamma_N^{\theta,*}) = \arg \min_{(\alpha, \gamma) : \Gamma_\tau(\mathbf{1}_\theta)} \{ \omega_p R(1 - \alpha) + \omega_k \bar{h}_2(\theta, \alpha, \gamma) + C_{\text{maint}}(\gamma, \alpha; \theta) \} \quad (33)$$

Si para algún tipo el conjunto factible degenerado $\Gamma_\tau(\mathbf{1}_\theta)$ es vacío, el benchmark respectivo se calcula sobre el mínimo no restringido en la malla.

Mecanismo de escape ante infactibilidad (Fallback)

Si el conjunto factible en un período τ dado es completamente vacío, es decir, $\Gamma_\tau(\mu_\tau) = \emptyset$ (ningún punto de la grilla 101×101 satisface simultáneamente las tres restricciones IC^K , IR^K y IR^F):

1. El solver desactiva temporalmente las restricciones y evalúa las funciones objetivo completas V_R y V_N sobre toda la grilla para hallar sus mínimos no restringidos:

$$idx_R^{unc} = \arg \min_{(\alpha, \gamma)} V_R(\alpha, \gamma), \quad V_R^{unc} = V_R(idx_R^{unc}) \quad (34)$$

$$idx_N^{unc} = \arg \min_{(\alpha, \gamma)} V_N(\alpha, \gamma), \quad V_N^{unc} = V_N(idx_N^{unc}) \quad (35)$$

2. Compara los valores mínimos no restringidos de ambas ramas. Si $V_R^{unc} \leq V_N^{unc}$, se selecciona la rama de Rescate, fijando los controles óptimos en la grilla correspondientes a idx_R^{unc} . En caso contrario, se selecciona la rama de Negociación, fijando los controles en idx_N^{unc} .
3. Retorna el indicador de viabilidad `feasible = False` para propósitos de reporte y visualización en el DataFrame, permitiendo que la simulación progrese continuamente sin colapsar por indeterminación matemática.

Términos de L_1^b

$$L_1^R(\theta, \alpha, \gamma) = \omega_k (1 - \tilde{p}_{\text{surv}}(\theta)) + C_{\text{ops}}(\gamma, \alpha; \theta), \quad (36)$$

$$L_1^N(\theta, \alpha, \gamma) = \omega_p R(1 - \alpha) + \omega_k \bar{h}_2(\theta, \alpha, \gamma) + C_{\text{maint}}(\gamma, \alpha; \theta), \quad (37)$$

con $C_{\text{ops}}(\gamma, \alpha; \theta) = c_0 + c_1 \gamma + \frac{c_2}{2} \gamma^2 + c_3 \alpha + \frac{c_4}{2} \alpha^2 + c_5 \gamma \alpha$ (análogo para C_{maint} con $m_0, \dots, m_5$); los coeficientes son específicos por tipo y **recalibrados** en `app_DL.py/train_captor_val` respecto a los valores crudos de `app.py` (véase §6.6, ítem sobre $C_{\text{ops}}/C_{\text{maint}}$).

6.2. Probabilidades como funciones de (α, γ)

Todas se recalculan, vectorizadas sobre la malla, en cada llamada al oráculo (función `outcome_probs_grid, p_cap_eff_grid, p_pay_eff_grid` de `train_captor_value_net`).

$$p_{det}(\theta, \alpha, \gamma) = \Lambda(\eta_0(\theta) + \eta_1\alpha + \eta_2\gamma), \quad (38)$$

$$idx_j(\theta, \alpha, \gamma) = \beta_{K,j}(\theta) + \beta_z \pm \zeta_\alpha(\theta)\alpha \pm \zeta_\gamma(\theta)\gamma \mp \zeta_d p_{det}, \quad j \in \{\text{pago, muerte, rescate}\}, \quad (39)$$

$$\lambda_j = M_\tau \cdot \lambda_{j,0} \cdot e^{idx_j}, \quad \bar{h}_j = (1 - e^{-\sum_j \lambda_j}) \cdot \frac{\lambda_j}{\sum_j \lambda_j}, \quad (40)$$

$$\tilde{p}_{cap}(\theta, \alpha, \gamma) = p_{rescue} \Lambda(\dots) + p_{nego} \Lambda(\dots) \quad (\text{Block F}), \quad (41)$$

$$\tilde{p}_{pay}(\theta, \alpha, \gamma) = p_{coop} h_1^{coop} + p_{col} h_1^{pay} \quad (\text{ramas hipotéticas “coop”/“pay”}). \quad (42)$$

M_τ se recalcula en cada τ vía $M_\tau = T_0 \max\{H_{ratio} e^{-\eta_{cal}\tau}, \underline{c}\}$, usando τ directamente (no la variable `t_days` de la pestaña 1, que aquí no se utiliza para este cómputo). $\Delta H_1(\alpha, \gamma)$ reutiliza exactamente el mecanismo de Bayes de 10 ramas (m, d) ya descrito para la fila ΔH de $\tau = 0$ (§5), evaluado en cada uno de los 10,201 puntos de la malla.

6.3. Las tres restricciones (conjunto factible $\Gamma_1(\mu_1)$)

Siguiendo `Bernal_H.tex` (eq:gamma-factible) y `Appendix_3.tex` (Definición del conjunto factible, Teorema de implementabilidad condicional), las tres condiciones se imponen **durante** la búsqueda (máscara booleana sobre la malla, con $+\infty$ en los puntos infactibles), no como verificación posterior:

$$IC^K : \min_{\theta_j} \sum_{\theta_i} \mu_1(\theta_i) [\text{mejor}(\theta_i) - U_{b(\theta_j)}(\theta_i)] \geq 0, \quad (43)$$

$$IR^K : \sum_{\theta} \mu_1(\theta) [U_{rel}(\theta) - \max\{V_{cont}(\theta), U_{kill}(\theta)\}] \geq 0, \quad (44)$$

$$IR^F : U_{coop}(\theta_F) \geq U_{col}(\theta_F), \quad U_{coop} = -\mathbb{E}_\mu[\tilde{p}_{pay}]R - e_\tau(\gamma, \theta_F), \quad U_{col} = -\mathbb{E}_\mu[\tilde{p}_{pay}]R(1 + \varrho). \quad (45)$$

$\Gamma_1(\mu_1) = IC^K \wedge IR^K \wedge IR^F$; el `argmin` de la ec. (28) solo puede caer en un punto con $\Gamma_1 = \text{verdadero}$.

6.4. La red de valor entrenada ($\hat{V}$)

$$V_{cont}(\theta, \alpha, \gamma) = \tilde{p}_{pay}R(1-\alpha) - C_\tau(\gamma, \theta) - \tilde{p}_{cap}F_{cap}(\theta) + \tilde{\beta}(\theta)(1-\tilde{p}_{cap}) \cdot \underbrace{\sum_{m,d} \Pr(m, d \mid \mu_\tau, \alpha, \gamma) \hat{V}(\theta, \mu_{\tau+1}^{m,d}, \tau+1)}_{\mathbb{E}[V_{\tau+1}]} \quad (46)$$

$\hat{V}$ es un MLP (CaptorValueNet, 2 capas ocultas de 96 neuronas, activación Tanh) con **34 variables de entrada**: μ (4), τ normalizado (1), θ *one-hot* (4), R (1), $\tilde{\beta}(\theta)$ (4), zona (5, *one-hot*), riqueza (2), víctima (2), tipo de Estado (2), T_{mad} , T_0 , η_{cal} , $\underline{c}$, H_{ratio} , λ_4 , η_1 , η_2 , β_R (9 escalares) — todos los parámetros ajustables de la pestaña 1 salvo los que se vuelven endógenos en la recursión (a_S_star , a_K_star , a_F_star , $iota_t$, $correct_id$, t_days ; véase §6.6).

Se entrenó por *backward induction* secuencial, $\tau = 10 \rightarrow 1$: en cada nivel, 2,000 puntos μ muestreados vía simulación Monte Carlo (con los 34 parámetros también muestreados dentro de sus rangos de *slider*) se resuelven con el oráculo (usando la red **ya entrenada y congelada** de $\tau+1$, o la condición terminal $\hat{V} \equiv 0$ en $\tau = T_{max}+1$), generando $V_\tau(\theta \mid \mu) = \max\{U_{rel}, U_{kill}, V_{cont}\}$ como etiqueta de regresión (pérdida MSE, Adam). No hay punto fijo ni iteración externa: cada nivel usa exclusivamente el nivel siguiente, ya resuelto.

Corrida realizada. $T = 10$, $N = 2,000$ puntos por nivel, 200 épocas de ajuste por nivel. Tiempo total de entrenamiento: ≈ 32 minutos (offline, una sola vez; no se repite en cada sesión de la app salvo que cambien los 34 parámetros estructurales usados para generar las etiquetas). Pérdida final por nivel: de 0.0069 ($\tau = 10$) a 0.0009 ($\tau = 1$), decreciente y estable en los 10 niveles.

6.5. Cálculo de cada variable de la Tabla 5.2 en $\tau = 1$

$\mu_1(\theta)$ Regla de Bayes de registro completo (Bernal_H.tex eq:bayes-update): $\mu_1(\theta) \propto \mu_0(\theta) \cdot \mathcal{L}_{I,K}(\theta) \cdot \Pr(m \mid \theta) \cdot \Pr(d \mid \theta) \cdot \mathcal{L}_C(\theta \mid V_t)$, usando m y d **realizados** (botones “Draw Physical Outcome” y “Draw Detection Signal” de la pestaña 1). Requisito previo al botón de optimización.

$a_1^{S*}, \alpha_1^*, \gamma_1^*$ Salida directa de `solve_state_problem` (ec. 28), comparando V_1^R vs. V_1^N ya restringidos por Γ_1 .

$H(\mu_1)$ $-\sum_\theta \mu_1(\theta) \ln \mu_1(\theta)$, calculado directamente de μ_1 (no requiere la red).

ΔH **Estado** ($\tau = 1$)

ΔH_1 evaluado en el punto (α_1^*, γ_1^*) ya encontrado (no en toda la malla; se extrae del mismo arreglo usado para restar $-\psi_H \Delta H_1$ del objetivo).

$\Gamma_t(\mu_t)$ **bajo EV** ($\tau = 1$)

El booleano `feasible` devuelto por el oráculo: verdadero si *algún* punto de la malla satisface $IC^K \wedge IR^K \wedge IR^F$ (en cuyo caso el punto elegido, por construcción, lo satisface); falso si $\Gamma_1 = \emptyset$ en toda la malla.

$IR^K(\theta_K)$ **tipo verdadero** ($\tau = 1$)

No es la versión ponderada por μ_1 usada en Γ_1 : se recalcula específicamente para el tipo verdadero (`cov_perp`), $U_{rel}(\theta^*) - \max\{V_{cont}(\theta^*), U_{kill}(\theta^*)\}$, en el mismo punto (α_1^*, γ_1^*) .

$\alpha_R^\mu, \gamma_R^\mu, \alpha_N^\mu, \gamma_N^\mu$ ($\tau = 1$)

Los mismos 16 benchmarks estructurales por tipo (siguiente ítem), reponderados por μ_1 en vez de μ_0 : $\alpha_R^\mu(\mu_1) = \sum_\theta \mu_1(\theta) \alpha_R^{\theta,*}$, etc.

$\gamma_R^{\theta,*}, \alpha_R^{\theta,*}, \gamma_N^{\theta,*}, \alpha_N^{\theta,*}$ (**16 filas**, $\tau = 1$)

Se reportan tras presionar el botón, como argmins **genuinamente recalculados** en $\tau = 1$ (no copiados de $\tau = 0$): rama rescate, $\arg\min_{(\alpha, \gamma)} \{\omega_K(1 - p_{surv}(\theta, \theta)) + C_{ops}(\gamma, \alpha; \theta)\}$ usando `cvn.p_surv_raw`; rama negociación, $\arg\min_{(\alpha, \gamma)} \{\omega_P R(1 - \alpha) + \omega_K \bar{h}_2(\alpha, \gamma; \theta, M_{\tau=1}) + C_{maint}(\gamma, \alpha; \theta)\}$ usando `cvn.outcome_probs` con $M_{\tau=1} = \text{cvn.m_t}(1, p)$ (el filtro de maduración específico de $\tau = 1$, no el M_t fijo de la pestaña 1). Verificado numéricamente que la rama rescate coincide exactamente con $\tau = 0$ (invarianza estructural correcta, no reutilización), mientras la rama negociación difiere (p.ej. $\gamma_N^{PAR,*}$: $0.64 \rightarrow 0.61$ con μ_{cal} de referencia), confirmando que ahora sí depende de τ .

6.6. Limitaciones documentadas explícitamente

- C_{ops}/C_{maint} **recalibrados**. Los coeficientes usados (interior, no de esquina) son una recalibración propia de esta app, distinta de los valores crudos de `app.py` — necesaria para que el óptimo de (α, γ) no colapse siempre a $(0, 0)$ o $(1, 0)$ (véase discusión en el historial de cambios de `app_DL.py`).
- ϱ (**prima de colusión**) **no calibrado en app.py**. Se fijó $\varrho = 2.0$ tras encontrar que $\varrho = 0$ hace IR^F **estructuralmente imposible** de satisfacer para cualquier $\phi_F, \nu_F > 0$ ($U_{coop} - U_{col} = -e_\tau < 0$ siempre). ϕ_F, ν_F, κ_F también se recalibraron a la baja por la misma razón.
- **Horizonte truncado** en $T = 10$, no $T = 100$. La red aproxima el problema de parada óptima del Secuestrador solo hasta ese horizonte; escalar a $T = 100$ (aprobado como próximo paso, pendiente de ejecución) requiere ≈ 9 –11 horas de entrenamiento offline con la misma configuración ($N = 2,000$ puntos/nivel).
- **Hallazgo de calibración, no error de código**. Con la calibración verificada contra `app.py` (κ_{rel} bajo frente a C_τ alto), “Liberar” domina a “Continuar” para los 4 tipos incluso con el valor de continuación dinámico ya incorporado — confirmado explorando el mejor caso posible dentro de los rangos de parámetros existentes (sin encontrar ningún punto donde “Continuar” gane).

7. VISUALIZACIONES DE DIAGNÓSTICO (PESTAÑA 3)

Gráfico	Datos graficados	Propósito
Convergencia de creencias	$\mu_t(\theta)$ para los 4 tipos vs. t ; línea vertical en <code>absorb_step</code>	Verificar si el Estado converge a identificar θ_K^* antes de la absorción
Trayectoria de instrumentos	α_t^*, γ_t^* vs. t	Evolución de interdicción y presión, ahora inferidas por la red
Incertidumbre y precisión	$H(\mu_t), \iota_t$ vs. t	Entropía (incertidumbre) vs. precisión (confianza)
Factibilidad de restricciones	IC^K, IR^F (1/0) vs. t	Verificación ex-post (nota: IC^K siempre 1, §4)
Convergencia de RegretNet (nuevo)	Pérdida social y arrepentimiento total agregado vs. época de entrenamiento	Diagnóstico de la fase de entrenamiento (§3): verificar que la red efectivamente reduce pérdida y arrepentimiento

8. DIAGRAMA DE FLUJO RESUMEN

Entrada

Sidebar (escenario, T , ψ_H , $\underline{c}$, calibración, hiperparámetros RegretNet) $\rightarrow$ priors (fijos / manuales / *model-driven* vía logit) $\rightarrow \mu_0$ normalizado.

Entrenamiento RegretNet

(fase única, punto fijo μ_0 , $t = T_{\text{horizon}}/2$, repetido por `epochs_training` épocas):

- (1) *Forward*: $\text{StateNetwork}(\mu_0, \theta_F, \theta_V, t) \rightarrow (\alpha_h, \gamma_h)$.
- (2) Pérdida social ponderada por μ_0 + subsidio de entropía (ec. (5)).
- (3) $\text{Regret}(\theta) = \max\{0, u_{\text{mentira}}(\theta) - u_{\text{honesto}}(\theta)\}$.
- (4) $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{social}} + \sum_{\theta} \lambda_{\theta} \text{Regret}(\theta) + \frac{\rho}{2} \sum_{\theta} \text{Regret}(\theta)^2$.
- (5) Paso de Adam (primal) + $\lambda_{\theta} \leftarrow \max\{0, \lambda_{\theta} + \rho \text{Regret}(\theta)\}$ (dual).

Inicialización del entorno

`env = KidnappingEnvironment( $T_{\text{mad}}$ ,  $\lambda_4$ ); mdg = MDGLayer( $T_0, \underline{c}, \eta_{\text{cal}}$ ).`

Bucle de simulación

`for t in range(T_horizon)` (aquí μ_t sí varía período a período):

- (1) $M(t) = \min\{1, (t/T_{\text{mad}})^2\}$.
- (2) $H(\mu_t), \iota_t$.
- (3) $T_t = T_0 \max\{(H(\mu_t)/H(\mu_0)) e^{-\eta_{\text{cal}} t}, \underline{c}\}$.
- (4) $(\alpha_t^*, \gamma_t^*) = \text{net}(\mu_t, \theta_F, \theta_V, t/150)$ — inferencia pura.
- (5) Rescatar vs. Negociar (p_{surv} logística).
- (6) Mejor respuesta del secuestrador real y de la familia.
- (7) Sorteo MDG (`mdg.sample_action`).
- (8) Riesgos en competencia con $M(t) \rightarrow$ outcome / absorción.
- (9) Actualización bayesiana de μ_t .
- (10) IR^K, IR^F, IC^K (*placeholder*, siempre verdadero) $\rightarrow$ se registra la fila.

Salida `df_sim(Thorizon filas)` → Tabla 5.2 transpuesta (primeros 15 períodos) → gráficos de diagnóstico + curva de convergencia de RegretNet (pestaña 3).